ME 305 Mechatronics

Department of Mechanical Engineering

Cal Poly

Lab 3 Report: Cooperative Multi-Tasking LED Blinker with Robust I/O Interface

Written by Gabe Choi and Maddie Howard

10/28/25

Overview/Objective:

The main objective of Lab 3 is to blink two different pairs of LED lights at different rates, specified from user input on the keypad. We needed to use cooperative multitasking with each task designed as a finite state machine. The tasks used in our code are outlined below as well as drawings of our finite state machines. The program was designed to accept magnitudes ranging from 1 to 65,535 with error messages to accompany a no digit input, 0-value input or a too large value input. For cooperative multitasking, each task only gets a little bit of time to run before moving to the next tasks, which needed to be applied to how we ran our display task and backspace key.

For the hardware, we used a delay routine of 1.00ms given to us in the lab manual. We wired the two electrical circuits so that each would drive two separate LEDs with two separate BS170 MOSFETs. As specified by the lab manual, bit_4 and bit_5 of PORT P drove the green and red LEDs of the first pair and bit_6 and bit_7 of PORT P drove the green and red LEDs of the second pair.

As for the software section of the lab, our program needed to continually check for if the F1 or F2 key had been pressed, with the F1 key corresponding to first pair of LEDs and the F2 key corresponding to the second pair of LEDs. If the F1 or F2 key had been pressed, our program needed to complete the following actions:

1. Turn OFF both LEDs in the appropriate LED pair.
2. Display an appropriate prompt on the LCD to give instructions to the user.
3. Accept characters representing decimal digits from the keypad.
4. Ignore all characters except digits 0-9, and the backspace and enter keys.
5. Echo valid digits to the LCD and store them as ASCII characters in a buffer.
6. Handle the backspace key appropriately.
7. The number entered by the user will be terminated by the enter key. Upon receiving a valid enter key, your program should convert the ASCII characters to a BCD number, then convert that BCD number to a binary number, and begin blinking the appropriate LED pair at the rate associated with the period entered.
8. Error conditions, such as no digits, zero is an inappropriate magnitude and magnitude too large should be handled properly, which includes the display of an appropriate error message. User friendliness is a required feature for your program.

9. Any time an LED pair is blinking, the period of that LED pair should be visible on the LCD, and the corresponding period should be blank if either LED pair is not blinking.

We were given shell code similar to code we had developed in lab previously or in our lecture assignments to start the lab.

Tasks:

Task #1: MasterMind

Mastermind is the brain of the whole code. It is in charge to direct and control all the other tasks to ensure that the overall objective of the program is running efficiently.

- State 0 – Initialization State. This includes all the hardware and software initialization for Mastermind. State 0 also sets the Time1, Time2 and F1 prompt and F2 prompt flags so that display will put those messages on the screen right at the beginning.
- State 1 – Waiting State. In state 1, the program will wait until all the initializations in state 0 are completed
- State 2 – Hub State. This state checks to see whether a struck key is waiting to be dealt with. If the key has been struck, the ASCII value is determined, and the next state of MasterMind is selected. If no keys have been struck, then the program returns to *main* and the next task is executed.
- State 3 – Digit State. From Hub, if the struck key is a digit, then the program will come to state 3. Digit state will only accept digits if a function key (F1 or F2) has been pressed. Digit state also checks to see if buffer is full and stores the digit in buffer if there is room. At the end, it returns to Hub.
- State 4 – Function State. From Hub, if the struck key is either F1 or F2, then the program will come to state 4. Depending on if the key if F1 or F2, the state will raise the F1 or F2 Flag, pause the corresponding LED pair and reset the digit counter. At the end, it returns to Hub.
- State 5 – Backspace State. From Hub, if the struck key is backspace, then the program will come to state 5. The state will check to see which line it is on, depending on which F1 or F2 Flag is true. This state raises the backspace flag and decrements the count depending on if any digits need to be deleted. Once the backspace flag gets cleared by Display, it returns to Hub.
- State 6 – Enter State. From Hub, if the struck key is entered then the program will come to state 6. First, the state checks to see if any digits have been entered. If not, the no digits error message flag is raised. Then it converts the ASCII values in buffer to binary and checks to make sure the value isn't too large or zero. Once the value is determined as valid, it stores the value in either Tick 1 or Tick 2 depending on if F1 or F2 has been pressed. Then it clears the F1 or F2 flag and it returns to Hub.

Task #2: Keypad Driver

The keypad driver monitors the keypad for any struck keys from the user and reports the ASCII value of each of the keys back to MasterMind.

- State 0 – Initialization State. This state clears the key flag raised by MasterMind and initializes the keypad hardware.
- State 1 – Key Press State. In state 1, the program checks to see if the LKEY_FLG has been pressed, meaning a key has been detected. If a key has been pressed, then it reads the ASCII value and adds it into the key buffer flag, which is a signal to MasterMind to process the digit.
- State 2 – Waiting State. Task 2 will stay in state 2 after detecting a key, until MasterMind has cleared the key flag and therefore acknowledge that task 2 can return to state 1 and wait for the next key.

Task #3: Display

The display task carries out any printing of messages or values on the LCD display. In the spirit of cooperative multitasking, it is important that the display task only prints one character on the display per run. This meant that our code needed to implement "handshakes" between tasks, so other tasks could run concurrently while the display was being updated.

- State 0 – Initialization State. This state initializes the LCD hardware.
- State 1 – Hub State. The Hub checks all the display flags and routes to which state is needed.
- State 2 – Display Message 1 State. State 2 oversees displaying the "TIME1 =" on the first line, corresponding to the first pair of LEDs.
- State 3 – Display Message 2 State. State 3 oversees displaying the "TIME2 =" on the second line, corresponding to the second pair of LEDs.
- State 4 – Display Message 3 State. State 4 oversees displaying the F1 message on the first line, corresponding to the first pair of LEDs.
- State 5 – Display Message 4 State. State 5 oversees displaying the F2 message on the second line, corresponding to the second pair of LEDs.
- State 6 – Echo State. This state determines which line to echo based on the active function key. Then it loads the digit from buffer and displays it at the cursor position on the screen.
- State 7 – Backspace State. This state determines which line to backspace based on the active function key. Then it goes through a sequence to complete the backspace operation, while allowing for the code to continue cooperative multitasking. Once it is complete, it will also clear the Backspace flag set by MasterMind.
- State 8 – Magnitude Too Large Message State. State 8 oversees displaying the magnitude too large error message if the magnitude too large message flag has been raised. The message will be displayed on either line 1 or line 2 depending on which function flag is raised.

- State 9 – Zero Magnitude Message State. State 9 oversees displaying the zero-magnitude error message if the zero-magnitude message flag has been raised. The message will be displayed on either line 1 or line 2 depending on which function flag is raised.
- State 10 – No Digits Entered Message State. State 10 oversees displaying the no digits entered error message if the no digits entered message flag has been raised. The message will be displayed on either line 1 or line 2 depending on which function flag is raised.

Task #4: Pattern 1

Pattern 1 runs the first set of LEDs and directs the flashing pattern for the red and green LEDs.

- State 0 – Initialization State. State 0 initializes Port P for LED control and turns off both LEDs. It also configures pins as outputs.
- State 1 – 1st Pattern Sequence. State 1 completes the first pattern sequence for LED pair 1, green on and red off. It then checks to see if the timing period is complete from the Done 1 flag.
- State 2 – 2nd Pattern Sequence. State 2 completes the second pattern sequence for LED pair 1, green and red off. It then checks to see if the timing period is complete from the Done 1 flag.
- State 3 – 3rd Pattern Sequence. State 3 completes the third pattern sequence for LED pair 1, green off and red on. It then checks to see if the timing period is complete from the Done 1 flag.
- State 4 – 4th Pattern Sequence. State 4 completes the fourth pattern sequence for LED pair 1, both green and red off. It then checks to see if the timing period is complete from the Done 1 flag.
- State 5 – 5th Pattern Sequence. State 5 completes the fifth pattern sequence for LED pair 1, both green on and red on. It then checks to see if the timing period is complete from the Done 1 flag.
- State 6 – 6th Pattern Sequence. State 6 completes the sixth pattern sequence for LED pair 1, both green on and red off. It then checks to see if the timing period is complete from the Done 1 flag.
- State 7 – No Timing Set State. State 7 is a bypass state if there is no timing set. In this case, the state keeps the LEDs turned off and returns to state 6 until the Ticks 1 is set.

Task #5: Timing 1

Timing 1 takes the user input value and runs the correct frequency for the first pair of LEDs.

- State 0 – Initialization State. State 0 initializes the timer and loads the period value from Ticks 1.
- State 1 – Active Countdown State. State 1 checks to see if the pattern has been completed or not. Once it has been completed the state advances the pattern sequence and reloads the counter.
- State 2 – Idle State. State 2 is the idle state when no timing has been configured.

Task #6: Pattern 2

Pattern 2 runs the second set of LEDs and directs the flashing pattern for the red and green LEDs; it is the exact same as Pattern 1 with the appropriate variables changed.

- State 0 – Initialization State. State 0 initializes Port P for LED control and turns off both LEDs. It also configures pins as outputs.
- State 1 – 1st Pattern Sequence. State 1 completes the first pattern sequence for LED pair 2, green on and red off. It then checks to see if the timing period is complete from the Done 2 flag.
- State 2 – 2nd Pattern Sequence. State 2 completes the second pattern sequence for LED pair 2, green and red off. It then checks to see if the timing period is complete from the Done 2 flag.
- State 3 – 3rd Pattern Sequence. State 3 completes the third pattern sequence for LED pair 2, green off and red on. It then checks to see if the timing period is complete from the Done 2 flag.
- State 4 – 4th Pattern Sequence. State 4 completes the fourth pattern sequence for LED pair 2, both green and red off. It then checks to see if the timing period is complete from the Done 2 flag.
- State 5 – 5th Pattern Sequence. State 5 completes the fifth pattern sequence for LED pair 2, both green on and red on. It then checks to see if the timing period is complete from the Done 2 flag.
- State 6 – 6th Pattern Sequence. State 6 completes the sixth pattern sequence for LED pair 2, both green on and red off. It then checks to see if the timing period is complete from the Done 2 flag.
- State 7 – No Timing Set State. State 7 is a bypass state if there is no timing set. In this case, the state keeps the LEDs turned off and returns to state 6 until the Ticks 2 is set.

Task #7: Timing 2

Timing 2 takes the user input value and runs the correct frequency for the second pair of LEDs; it is the exact same as Pattern 2 with the appropriate variables changed.

- State 0 – Initialization State. State 0 initializes the timer and loads the period value from Ticks 2.
- State 1 – Active Countdown State. State 1 checks to see if the pattern has been completed or not. Once it has been completed the state advances the pattern sequence and reloads the counter.
- State 2 – Idle State. State 2 is the idle state when no timing has been configured.

Task #8: Delay

The last task, Delay, runs the delay subroutine which is programmed to delay for ~1.00ms.

- State 0 – Initialization State. There is no required initialization for the delay, so this state just advances the task to state 1.

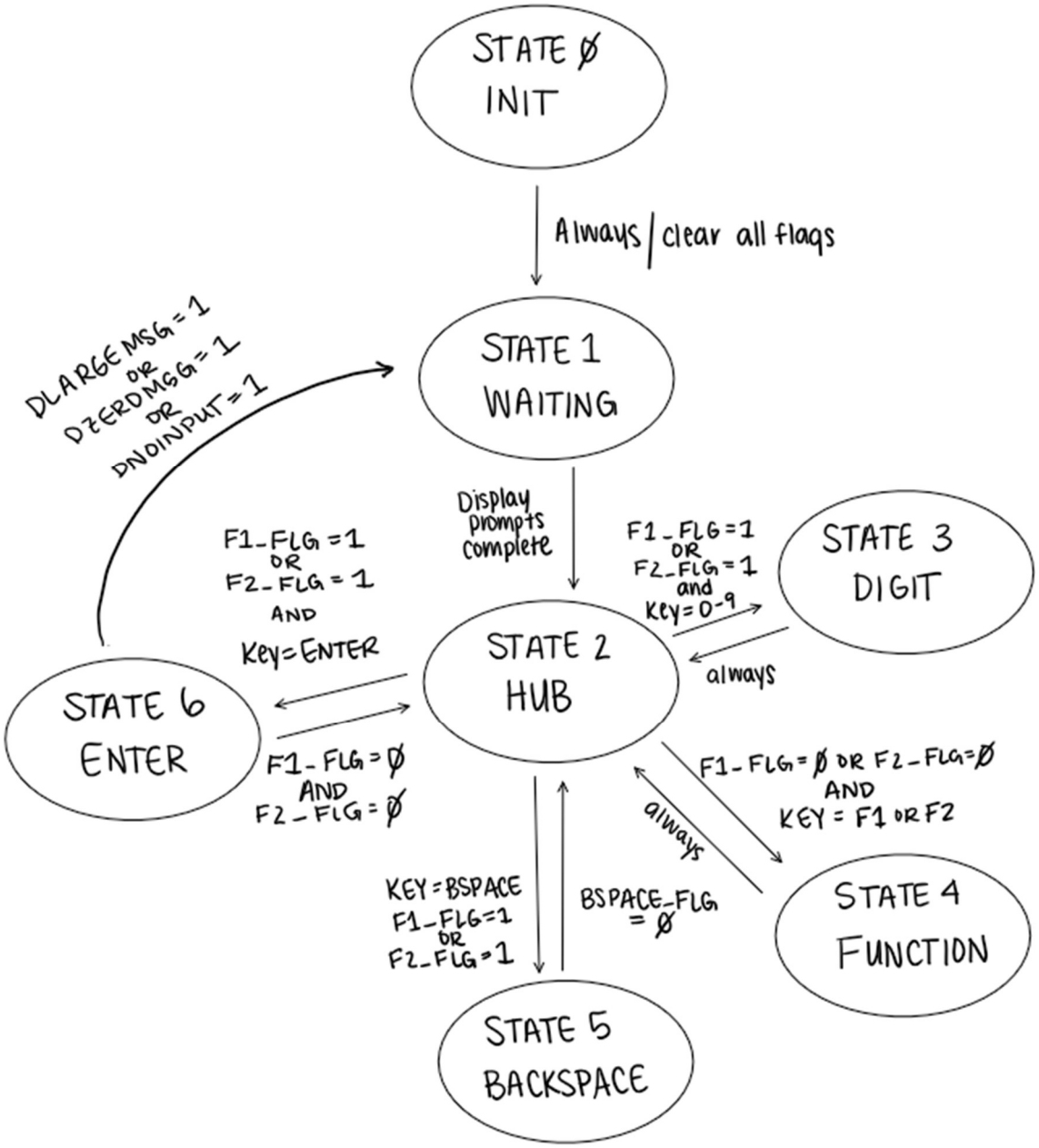
- State 1 – Generate Delay State. This state runs the delay subroutine.

Inter-task Communication Variables:

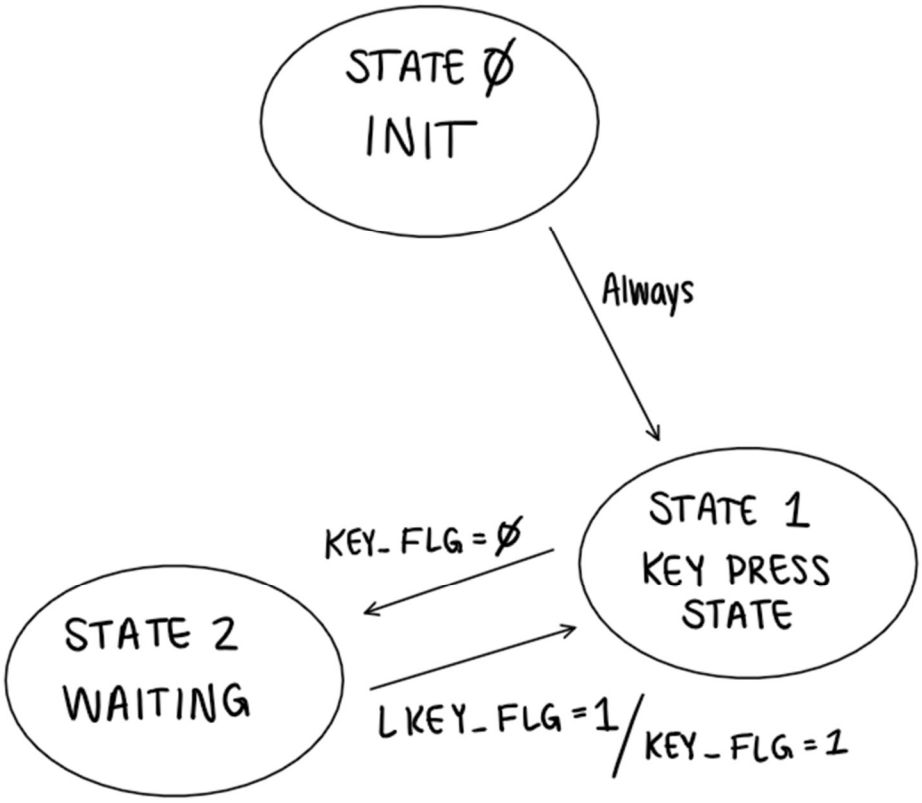
Variable Name	Variable Description	Task and State Set	Task and State Cleared
KEY_FLG	Key press detected	Task 2 State 1	Task 1 State 2
F1_FLG	F1 pressed (LED pair 1 and first line of display)	Task 1 State 4	Task 1 State 6
F2_FLG	F2 pressed (LED pair 2 and second line of display)	Task 1 State 4	Task 1 State 6
BSPACE_FLG	Backspace key pressed	Task 1 State 5	Task 3 State 7
DIGIT_FLG	Valid digit entered	Task 1 State 3	Task 3 State 6
DTIME1	Display “TIME1 =” message	Task 1 State 0	Task 3 State 2
DTIME2	Display “TIME2 =” message	Task 1 State 0	Task 3 State 3
DF1PRMPT	Display F1 prompt	Task 1 State 0	Task 3 State 4
DF2PRMPT	Display F2 prompt	Task 1 State 0	Task 3 State 5
TICKS_1	Period of LED pair 1	Task 1 State 6	Task 1 State 4
DONE_1	Countdown complete for pattern sequence	Task 5 State 1 Task 5 State 2	Task 5 State 0
TICKS_2	Period of LED pair 2	Task 1 State 6	Task 1 State 4
DONE_2	Countdown complete for pattern sequence	Task 7 State 1 Task 7 State 2	Task 7 State 0
DLARGEMSG	Display “magnitude too large” error message	Task 1 State 6	Task 3 State 8
DZEROMSG	Display “zero magnitude” error message	Task 1 State 6	Task 3 State 9
DNOINPUT	Display “no input” error message	Task 1 State 6	Task 3 State 10

Finite State Machines:

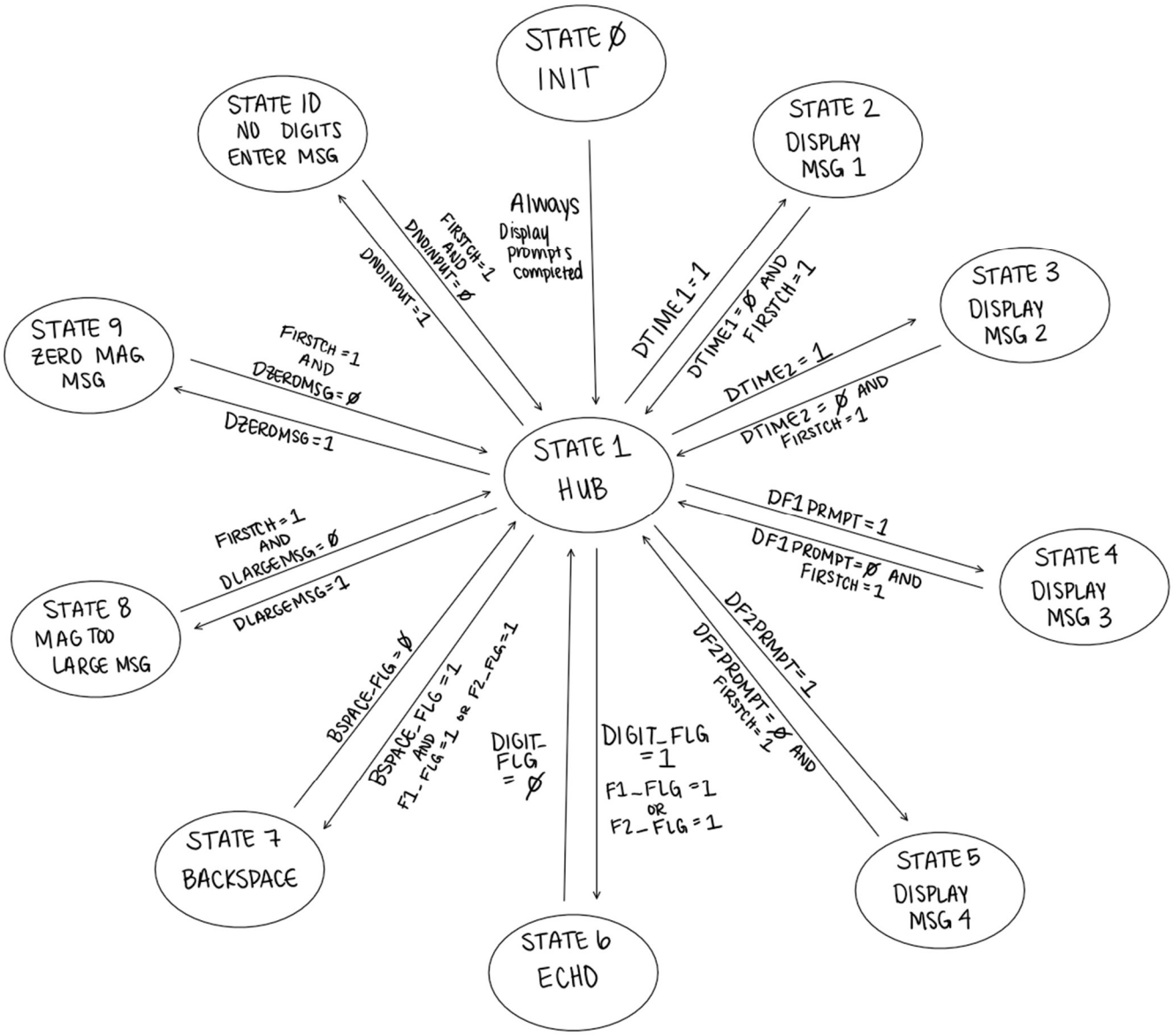
Task #1: MasterMind



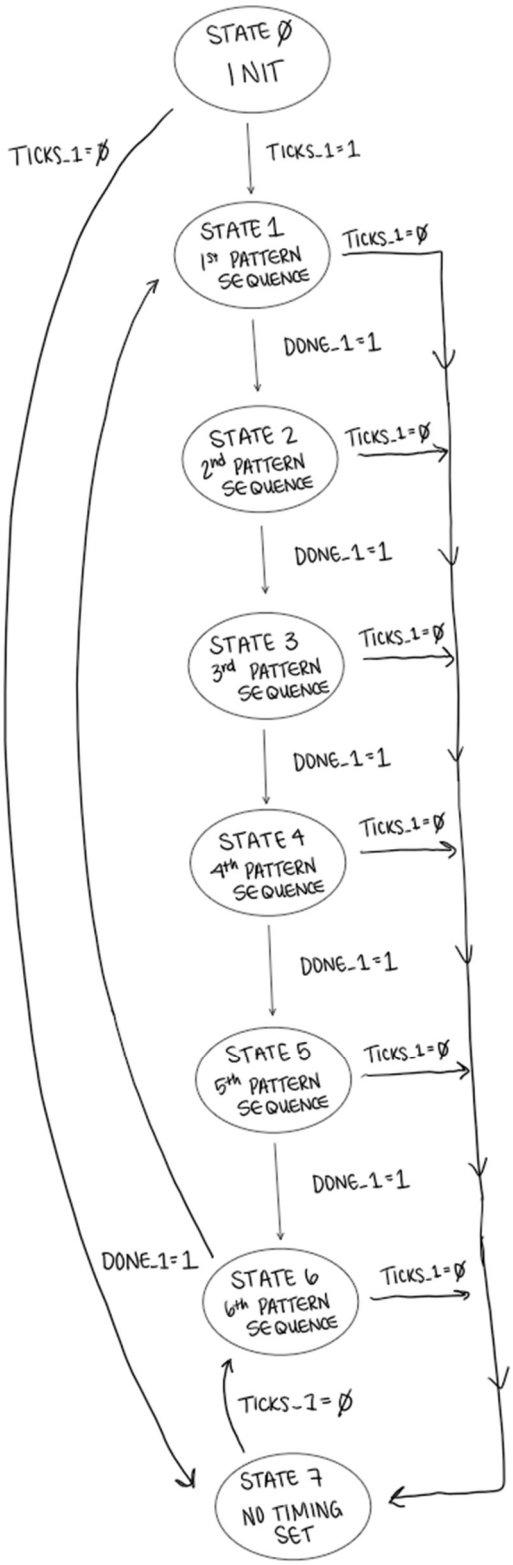
Task #2: Keypad Driver



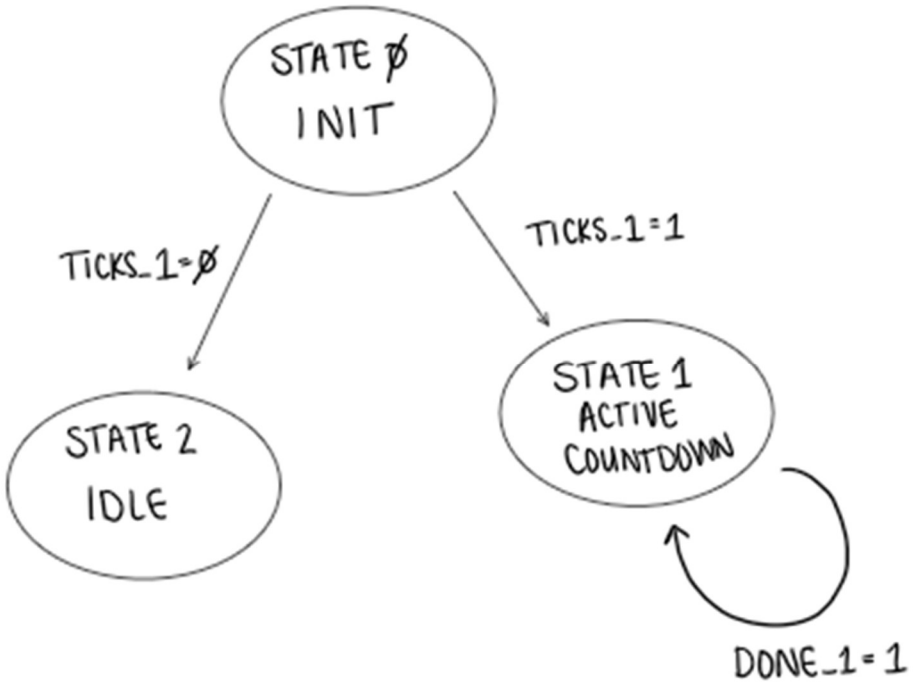
Task #3: Display



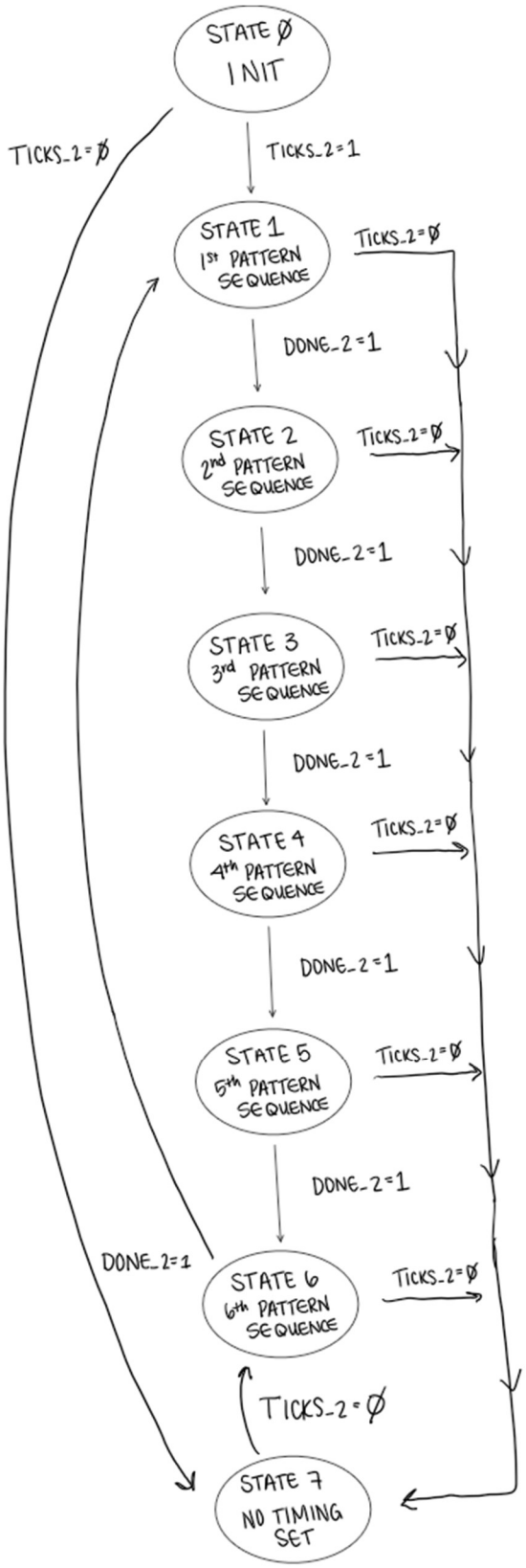
Task #4: Pattern 1



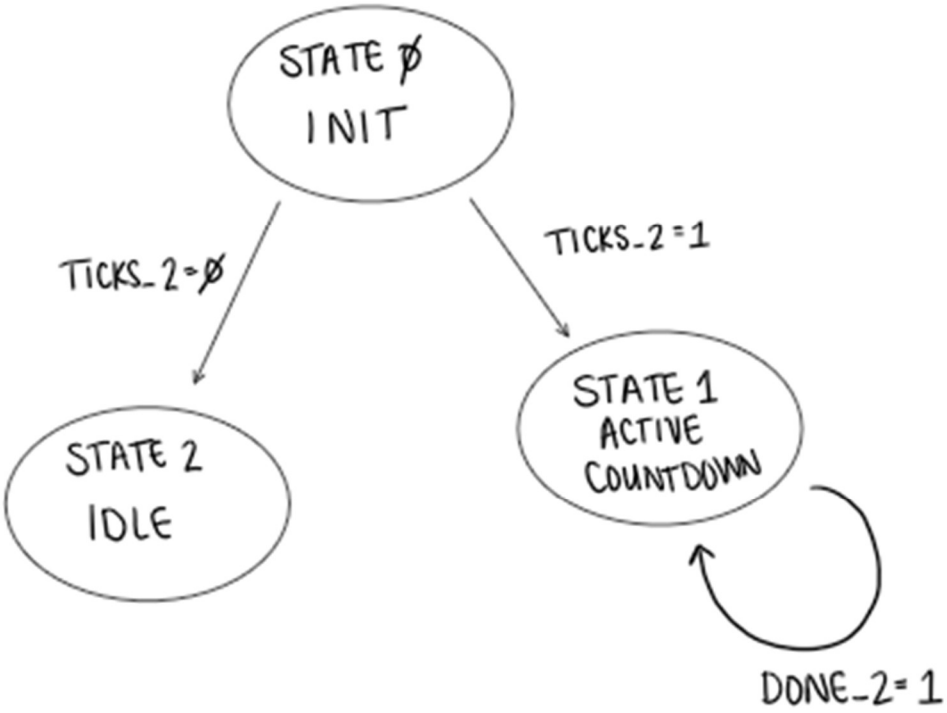
Task #5: Timing 1



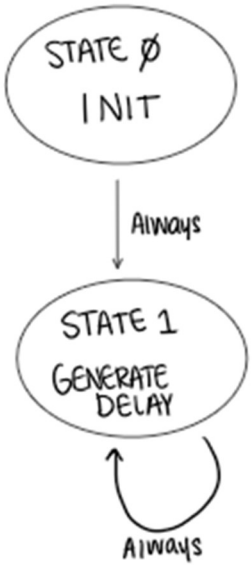
Task #6: Pattern 2



Task #7: Timing 2



Task #8: Delay



```
;*****************************************************************************
***
;* ME 305 Lab 3 - Cooperative Multitasking LED Blinker with Keypad/LCD I/O
*
;*****************************************************************************
***
;* Summary:
 *
;*   Implements cooperative multitasking to control two independent LED pairs with
*
;*   user-configurable blink periods. Users input timing values via keypad, which
*
;*   are displayed on LCD with appropriate prompts and error messages. The system
*
;*   validates input and converts ASCII digits to binary for timing control.
*
;*
 *
;* Author:
 *
;*   Gabe Choi and Maddie Howard
 *
;*   Cal Poly University
 *
;*   Fall 2025
 *
;*
 *
;* Revision History:
 *
;*   -                                                                    *
;*****************************************************************************
***

;/-----------------------------------------------------------------------------
--\
;| Include all associated files
|
;\-----------------------------------------------------------------------------
--/

;/-----------------------------------------------------------------------------
--\
;| External Definitions
|
;\-----------------------------------------------------------------------------
--/
 XDEF main

;/-----------------------------------------------------------------------------
```

```
--\
;| External References
|
;\--------------------------------------------------------------------------------
--/
 XREF ENABLE_MOTOR, STARTUP_MOTOR, UPDATE_MOTOR, DISABLE_MOTOR
 XREF STARTUP_PWM, STARTUP_ENCODER, READ_ENCODER, DISABLE_ENCODER
 XREF OUTDACA, OUTDACB
 XREF INITLCD, CLRSCREEN, SETADDR, GETADDR, CURSOR_ON, DISP_OFF
 XREF OUTCHAR, OUTCHAR_AT, OUTSTRING, OUTSTRING_AT
 XREF INITKEY, LKEY_FLG, GETCHAR
 XREF Entry, ISR_KEYPAD

;/--------------------------------------------------------------------------------
--\
;| Assembler Equates
|
;\--------------------------------------------------------------------------------
--/
; ASCII key codes
F1                  EQU     $F1             ; F1 function key
F2                  EQU     $F2             ; F2 function key
BSPACE              EQU     $08             ; Backspace key
ENT                 EQU     $0A             ; Enter key

; Port P configuration for LED control
PORTP               EQU     $0258           ; Port P data register
DDRP                EQU     $025A           ; Port P data direction register

; LED pair 1 bit masks (upper byte of Port P)
G_LED_1             EQU     %01000000       ; Green LED (bit 6)
R_LED_1             EQU     %10000000       ; Red LED (bit 7)
LED_MSK_1           EQU     %11000000       ; Both LEDs (bits 6-7)

; LED pair 2 bit masks (middle nibble of Port P)
G_LED_2             EQU     %00010000       ; Green LED (bit 4)
R_LED_2             EQU     %00100000       ; Red LED (bit 5)
LED_MSK_2           EQU     %00110000       ; Both LEDs (bits 4-5)

;/--------------------------------------------------------------------------------
--\
;| Variables in RAM
|
;\--------------------------------------------------------------------------------
--/
DEFAULT_RAM: SECTION

; Task state variables
t1state:            DS.B    1               ; MasterMind state
t2state:            DS.B    1               ; KeypadDriver state
```

```
t3state:              DS.B    1               ; Display state
t4state:              DS.B    1               ; Pattern_1 state
t5state:              DS.B    1               ; Timing_1 state
t6state:              DS.B    1               ; Pattern_2 state
t7state:              DS.B    1               ; Timing_2 state
t8state:              DS.B    1               ; Delay state

; Keypad communication variables
KEY_FLG:              DS.B    1               ; Flag: key press detected (set by
KeypadDriver)
KEY_BUF:              DS.B    1               ; Buffer: ASCII value of pressed
key

; Function key status flags
F1_FLG:               DS.B    1               ; Flag: F1 pressed (LED pair 1
selected)
F2_FLG:               DS.B    1               ; Flag: F2 pressed (LED pair 2
selected)

; Input handling flags
BSPACE_FLG:           DS.B    1               ; Flag: backspace pressed
DIGIT_FLG:            DS.B    1               ; Flag: valid digit entered

; Display control variables
FIRSTCH:              DS.B    1               ; Flag: displaying first char of
string (initialized to 1)
DPTR:                 DS.W    1               ; Pointer: address of next
character to display

; Display message flags
DTIME1                DS.B    1               ; Flag: display "TIME1 = " message
DTIME2                DS.B    1               ; Flag: display "TIME2 = " message
DF1PRMPT              DS.B    1               ; Flag: display F1 prompt
DF2PRMPT              DS.B    1               ; Flag: display F2 prompt

; Input buffer and counter
BUFFER:               DS.B    5               ; Buffer: stores up to 5 digit
characters
COUNT:                DS.B    1               ; Counter: number of digits in
BUFFER

; LED pair 1 timing variables
TICKS_1               DS.W    1               ; Period for LED pair 1 [ms] (0 =
awaiting input)
COUNT_1               DS.W    1               ; Countdown timer for LED pair 1
[ms]
DONE_1                DS.B    1               ; Flag: countdown complete, advance
pattern state

; LED pair 2 timing variables
```

```
TICKS_2               DS.W    1                  ; Period for LED pair 2 [ms] (0 =
awaiting input)
COUNT_2               DS.W    1                  ; Countdown timer for LED pair 2
[ms]
DONE_2                DS.B    1                  ; Flag: countdown complete, advance
pattern state

; Error message control
ERROR_TIME            DS.W    1                  ; Duration to display error
messages [ms]
DLARGEMSG             DS.B    1                  ; Flag: display "magnitude too
large" error
DZEROMSG              DS.B    1                  ; Flag: display "zero magnitude"
error
DNOINPUT              DS.B    1                  ; Flag: display "no input" error

;/-------------------------------------------------------------------------------
--\
;| Main Program Code
|
;\-------------------------------------------------------------------------------
--/
MyCode: SECTION

main:
    ; Initialize all task states to zero (initialization state)
    clr     t1state
    clr     t2state
    clr     t3state
    clr     t4state
    clr     t5state
    clr     t6state
    clr     t7state
    clr     t8state

top:
    ; Cooperative multitasking loop - each task executes one state per cycle
    jsr     TASK_1                          ; MasterMind: input processing &
control logic
    jsr     TASK_2                          ; KeypadDriver: key scanning
    jsr     TASK_3                          ; Display: LCD output management
    jsr     TASK_4                          ; Pattern_1: LED pair 1 blinking
pattern
    jsr     TASK_5                          ; Timing_1: LED pair 1 countdown
timer
    jsr     TASK_6                          ; Pattern_2: LED pair 2 blinking
pattern
    jsr     TASK_7                          ; Timing_2: LED pair 2 countdown
timer
    jsr     TASK_8                          ; Delay: 1ms system tick generation
```

```
    bra     top                                 ; Repeat forever

;===============================================================================
=====
;| TASK_1: MasterMind
   |
;|   Central controller that processes keypad input, manages state transitions,
   |
;|   validates user input, and coordinates between keypad, display, and LED tasks
   |
;===============================================================================
=====
TASK_1:
    ; State dispatcher
    ldaa    t1state
    beq     t1state0                            ; State 0: Initialization
    deca
    beq     t1state1                            ; State 1: Wait for initial display
    deca
    lbeq    t1state2                            ; State 2: Hub - decode keypad input
    deca
    lbeq    t1state3                            ; State 3: Digit entry handler
    deca
    lbeq    t1state4                            ; State 4: Function key handler
    deca
    lbeq    t1state5                            ; State 5: Backspace handler
    deca
    lbeq    t1state6                            ; State 6: Enter key handler
    rts                                         ; Undefined state - do nothing

t1state0:                                       ; Initialization
    ; Clear all flags
    clr     KEY_FLG
    clrw    TICKS_1
    clrw    TICKS_2
    clr     F1_FLG
    clr     F2_FLG
    clr     BSPACE_FLG
    clr     DIGIT_FLG
    clr     DLARGEMSG
    clr     DZEROMSG
    clr     DNOINPUT

    ; Initialize display timing and messages
    movw    #$300, ERROR_TIME                   ; Set initial display duration
    movb    #$01, DTIME1                        ; Request TIME1 display
    movb    #$01, DTIME2                        ; Request TIME2 display
    movb    #$01, DF1PRMPT                      ; Request F1 prompt display
    movb    #$01, DF2PRMPT                      ; Request F2 prompt display
    movb    #$01, t1state                       ; Advance to wait state
```

```
    rts

t1state1:                                   ; Wait for initial display completion
    tstw    ERROR_TIME                      ; Check if wait period expired
    beq     t1s1a                           ; If expired, proceed
    decw    ERROR_TIME                      ; Otherwise decrement timer
    rts

t1s1a:
    ; Transition to hub state and reset for next error message
    movb    #$02, t1state                   ; Advance to hub state
    movw    #3000, ERROR_TIME               ; Set error message display duration
(3 seconds)

    ; Check if F1 or F2 was pressed to refresh their prompts
    tst     F1_FLG
    bne     t1s1b
    bra     t1s1c

t1s1b:                                      ; F1 was pressed - refresh F1 prompt
    movb    #$01, DF1PRMPT
    clr     F1_FLG
    rts

t1s1c:                                      ; F2 was pressed - refresh F2 prompt
    movb    #$01, DF2PRMPT
    clr     F2_FLG
    rts

t1state2:                                   ; Hub - decode keyboard input
    tst     KEY_FLG                         ; Check if key is available
    bne     t1s2a
    rts                                     ; No key available - exit

t1s2a:
    ; Decode the key and branch to appropriate handler
    ldab    KEY_BUF

    cmpb    #F1                             ; Check for F1 key
    bne     t1s2b
    movb    #$04, t1state                   ; Route to function key handler
    bra     exit_t1s2

t1s2b:
    cmpb    #F2                             ; Check for F2 key
    bne     t1s2c
    movb    #$04, t1state                   ; Route to function key handler
    bra     exit_t1s2

t1s2c:
```

```
    cmpb    #BSPACE                         ; Check for backspace
    bne     t1s2d
    movb    #$05, t1state                   ; Route to backspace handler
    bra     exit_t1s2

t1s2d:
    cmpb    #ENT                            ; Check for enter
    bne     t1s2e
    movb    #$06, t1state                   ; Route to enter handler
    bra     exit_t1s2

t1s2e:
    ; Validate digit range (ASCII '0' to '9')
    cmpb    #$39                            ; Compare to '9'
    bhi     exit_t1s2                       ; Invalid if higher
    cmpb    #$30                            ; Compare to '0'
    blo     exit_t1s2                       ; Invalid if lower
    movb    #$03, t1state                   ; Valid digit - route to digit
handler

exit_t1s2:
    clr     KEY_FLG                         ; Acknowledge key processed
    rts

t1state3:                                   ; Digit entry handler
    ; Only accept digits if a function key (F1 or F2) has been pressed
    ldaa    F1_FLG
    oraa    F2_FLG                          ; Check if either function key active
    bne     t1s3a
    bra     exit_t1s3                       ; No function key - ignore digit

t1s3a:
    ; Check if buffer is full (max 5 digits)
    ldaa    COUNT
    cmpa    #$05
    beq     exit_t1s3                       ; Buffer full - ignore digit

    ; Store digit in buffer
    ldx     #BUFFER
    ldab    KEY_BUF
    stab    A,X                             ; Store at position COUNT
    inc     COUNT                           ; Increment digit counter
    movb    #$01, DIGIT_FLG                 ; Signal Display to echo digit

exit_t1s3:
    movb    #$02, t1state                   ; Return to hub
    rts

t1state4:                                   ; Function key (F1/F2) handler
    ; Only process if no function key currently active
```

```
     ldaa    F1_FLG
     oraa    F2_FLG
     bne     exit_t1s4                          ; Function key already active -
ignore

    ; Determine which function key was pressed
    ldaa    KEY_BUF
    cmpa    #F1
    beq     t1s4a
    cmpa    #F2
    beq     t1s4b
    movb    #$02, t1state                      ; Neither - return to hub
    rts

t1s4a:                                         ; F1 pressed - select LED pair 1
    movb    #$01, F1_FLG                       ; Activate F1 mode
    movb    #$01, DF1PRMPT                     ; Clear display line for input
    clrw    TICKS_1                            ; Pause LED pair 1
    clr     COUNT                              ; Reset digit counter
    movb    #$02, t1state                      ; Return to hub
    rts

t1s4b:                                         ; F2 pressed - select LED pair 2
    movb    #$01, F2_FLG                       ; Activate F2 mode
    movb    #$01, DF2PRMPT                     ; Clear display line for input
    clrw    TICKS_2                            ; Pause LED pair 2
    clr     COUNT                              ; Reset digit counter
    movb    #$02, t1state                      ; Return to hub
    rts

exit_t1s4:
    movb    #$02, t1state                      ; Return to hub
    rts

t1state5:                                      ; Backspace handler
    ; Check if backspace already processed (prevents multiple backspaces)
    tst     BSPACE_FLG
    bne     t1s5b

    ; Only process if a function key is active
    ldaa    F1_FLG
    oraa    F2_FLG/
    bne     t1s5a
    bra     exit_t1s5                          ; No function key active - ignore

t1s5a:
    movb    #$01, BSPACE_FLG                   ; Signal Display to show backspace
    rts

t1s5b:                                         ; Backspace complete - update counter
```

```
    movb    #$02, t1state                   ; Return to hub
    tst     COUNT                           ; Check if any digits to delete
    beq     exit_t1s5
    dec     COUNT                           ; Decrement digit counter

exit_t1s5:
    movb    #$02, t1state                   ; Return to hub
    rts

t1state6:                                   ; Enter key handler - validate and
apply input
    ; Only process if a function key is active
    ldaa    F1_FLG
    ORAA    F2_FLG
    bne     t1s6a
    bra     exit_t1s6                       ; No function key active - ignore

t1s6a:
    ; Check if any digits were entered
    ldaa    COUNT
    tsta
    beq     t1s6b                           ; No digits - error

    ; Convert ASCII buffer to binary value
    jsr     ASCII_2_BIN                     ; Result in X, error code in A

    ; Check error codes
    cmpa    #$00
    beq     t1s6c                           ; No error - accept value
    cmpa    #$01
    beq     t1s6d                           ; Error: value too large
    cmpa    #$02
    beq     t1s6e                           ; Error: value is zero

t1s6b:                                      ; Error: no digits entered
    movb    #$01, DNOINPUT                  ; Display error message
    movb    #$01, t1state                   ; Return to wait state (for error
display)
    rts

t1s6c:                                      ; Valid input - store in appropriate
TICKS variable
    tst     F1_FLG
    bne     t1s6ci
    tst     F2_FLG
    bne     t1s6cii

t1s6ci:                                     ; Store value for LED pair 1
    stx     TICKS_1
    movb    #$02, t1state                   ; Return to hub
```

```
    clr     F1_FLG                          ; Deactivate F1 mode
    rts

t1s6cii:                                    ; Store value for LED pair 2
    stx     TICKS_2
    movb    #$02, t1state                   ; Return to hub
    clr     F2_FLG                          ; Deactivate F2 mode
    rts

t1s6d:                                      ; Error: value exceeds maximum
(65535)
    movb    #$01, DLARGEMSG                 ; Display error message
    movb    #$01, t1state                   ; Return to wait state (for error
display)
    rts

t1s6e:                                      ; Error: value is zero
    movb    #$01, DZEROMSG                  ; Display error message
    movb    #$01, t1state                   ; Return to wait state (for error
display)
    rts

exit_t1s6:
    movb    #$02, t1state                   ; Return to hub
    rts

;=============================================================================
=====
;| TASK_2: KeypadDriver
   |
;|   Scans the keypad matrix for key presses and communicates with MasterMind
   |
;=============================================================================
=====
TASK_2:
    ; State dispatcher
    ldaa    t2state
    beq     t2state0                        ; State 0: Initialization
    deca
    beq     t2state1                        ; State 1: Checking for key press
    deca
    beq     t2state2                        ; State 2: Waiting for key
acknowledgment
    rts

t2state0:                                   ; Initialization
    clr     KEY_FLG                         ; Clear communication flag
    jsr     INITKEY                         ; Initialize keypad hardware
    movb    #$01, t2state                   ; Advance to checking state
    rts
```

```
t2state1:                                       ; Scan for key press
    tst     LKEY_FLG                            ; Check library key flag
    bne     t2s1a                               ; Key detected - process it
    rts                                         ; No key - exit

t2s1a:
    jsr     GETCHAR                             ; Read ASCII value into B
    stab    KEY_BUF                             ; Store in communication buffer
    movb    #01, KEY_FLG                        ; Signal MasterMind
    movb    #02, t2state                        ; Advance to wait state
    rts

t2state2:                                       ; Wait for MasterMind acknowledgment
    tst     KEY_FLG                             ; Check if MasterMind cleared flag
    beq     t2s2a                               ; Cleared - ready for next key
    rts                                         ; Still set - continue waiting

t2s2a:
    movb    #01, t2state                        ; Return to checking state
    rts

;===============================================================================
=====
;| TASK_3: Display
   |
;|   Manages all LCD output including prompts, user input echo, and error messages
   |
;===============================================================================
=====
TASK_3:
    ; State dispatcher
    ldaa    t3state
    beq     t3state0                            ; State 0: Initialization
    deca
    beq     t3state1                            ; State 1: Hub - check display flags
    rts

t3state0:                                       ; Initialization
    jsr     INITLCD                             ; Initialize LCD hardware
    movb    #$01, t3state                       ; Advance to hub state
    movb    #01, FIRSTCH                        ; Set first character flag
    rts

t3state1:                                       ; Hub - check all display flags and
route
    tst     DTIME1
    bne     t3state2                            ; Display TIME1 message
    tst     DTIME2
    bne     t3state3                            ; Display TIME2 message
```

```
    tst     DF1PRMPT
    bne     t3state4                        ; Display F1 prompt
    tst     DF2PRMPT
    lbne    t3state5                        ; Display F2 prompt
    tst     DIGIT_FLG
    lbne    t3state6                        ; Echo digit
    tst     BSPACE_FLG
    lbne    t3state7                        ; Echo backspace
    tst     DLARGEMSG
    lbne    t3state8                        ; Display "too large" error
    tst     DZEROMSG
    lbne    t3state9                        ; Display "zero" error
    tst     DNOINPUT
    lbne    t3state10                       ; Display "no input" error
    rts                                     ; No flags set - exit

t3state2:                                   ; Display "TIME1 = " message (top
line)
    ldaa    FIRSTCH
    cmpa    #$01                            ; Check if first character
    bne     t3s2a                           ; Not first - continue display

    ; First character - initialize display
    ldx     #TIME1                          ; Load message address
    ldaa    #$00                            ; LCD address: top-left corner
    jsr     PUTCHAR_1ST                     ; Initialize and display first char
    bra     bh_t3s2

t3s2a:
    jsr     PUTCHAR                         ; Display next character

bh_t3s2:
    tst     FIRSTCH                         ; Check if message complete
    beq     t3s2b                           ; Not complete - exit
    clr     DTIME1                          ; Message complete - clear flag
    movb    #$01, t3state                   ; Return to hub

t3s2b:
    rts

t3state3:                                   ; Display "TIME2 = " message (bottom
line)
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s3a

    ldx     #TIME2                          ; Load message address
    ldaa    #$40                            ; LCD address: bottom-left corner
    jsr     PUTCHAR_1ST
    bra     bh_t3s3
```

```
t3s3a:
    jsr     PUTCHAR

bh_t3s3:
    tst     FIRSTCH
    beq     t3s3b
    clr     DTIME2
    movb    #$01, t3state

t3s3b:
    rts

t3state4:                                   ; Display F1 prompt (top line
continuation)
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s4a

    ldx     #F1PRMPT                        ; Load prompt address
    ldaa    #$08                            ; LCD address: top line, after "TIME1
= "
    jsr     PUTCHAR_1ST
    bra     bh_t3s4

t3s4a:
    jsr     PUTCHAR

bh_t3s4:
    tst     FIRSTCH
    beq     t3s4b
    clr     DF1PRMPT
    movb    #$01, t3state

t3s4b:
    rts

t3state5:                                   ; Display F2 prompt (bottom line
continuation)
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s5a

    ldx     #F2PRMPT                        ; Load prompt address
    ldaa    #$48                            ; LCD address: bottom line, after
"TIME2 = "
    jsr     PUTCHAR_1ST
    bra     bh_t3s5

t3s5a:
```

```
    jsr     PUTCHAR

bh_t3s5:
    tst     FIRSTCH
    beq     t3s5b
    clr     DF2PRMPT
    movb    #$01, t3state

t3s5b:
    rts

t3state6:                                   ; Echo digit at cursor position
    ; Determine which line to echo to based on active function key
    tst     F1_FLG
    bne     t3s6a                           ; Echo to top line
    tst     F2_FLG
    bne     t3s6b                           ; Echo to bottom line
    bra     t3s6c                           ; No function key - shouldn't happen

t3s6a:                                      ; Echo to top line (F1 active)
    ldaa    COUNT                           ; Get digit position
    deca                                    ; Adjust for 0-based indexing
    ldx     #BUFFER
    ldab    A,X                             ; Load digit from buffer
    adda    #$08                            ; Calculate LCD address (top line +
8)
    jsr     OUTCHAR_AT                      ; Display at position
    bra     bh_t3s6

t3s6b:                                      ; Echo to bottom line (F2 active)
    ldaa    COUNT
    deca
    ldx     #BUFFER
    ldab    A,X
    adda    #$48                            ; Calculate LCD address (bottom line
+ 8)
    jsr     OUTCHAR_AT
    bra     bh_t3s6

bh_t3s6:
    clr     DIGIT_FLG                       ; Clear echo flag
    movb    #$01, t3state                   ; Return to hub

t3s6c:
    rts

t3state7:                                   ; Echo backspace sequence
    ; Determine which line based on active function key
    tst     F1_FLG
    bne     t3s7a                           ; Backspace on top line
```

```
    tst     F2_FLG
    bne     t3s7b                           ; Backspace on bottom line
    bra     t3s7c                           ; No function key - exit

t3s7a:                                      ; Backspace on top line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s7ai

    ldaa    COUNT                           ; Calculate cursor position
    ldab    #$07                            ; Base address - 1
    aba
    ldx     #BSPACE                         ; Load backspace sequence address
    jsr     PUTCHAR_1ST
    bra     bh_t3s7

t3s7ai:
    jsr     PUTCHAR
    bra     bh_t3s7

t3s7b:                                      ; Backspace on bottom line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s7bi

    ldaa    COUNT
    ldab    #$47                            ; Base address - 1
    aba
    ldx     #BSPACE
    jsr     PUTCHAR_1ST
    bra     bh_t3s7

t3s7bi:
    jsr     PUTCHAR

bh_t3s7:
    tst     FIRSTCH                         ; Check if backspace sequence
complete
    beq     t3s7c
    clr     BSPACE_FLG
    movb    #$01, t3state

t3s7c:
    rts

t3state8:                                   ; Display "MAGNITUDE TOO LARGE" error
    tst     F1_FLG
    bne     t3s8a                           ; Display on top line
    tst     F2_FLG
    bne     t3s8b                           ; Display on bottom line
```

```
    bra     t3s8c

t3s8a:                                      ; Display on top line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s8ai

    ldx     #LARGEMSG
    ldaa    #$08                            ; Position after "TIME1 = "
    jsr     PUTCHAR_1ST
    bra     bh_t3s8a

t3s8ai:
    jsr     PUTCHAR

bh_t3s8a:
    tst     FIRSTCH
    beq     t3s8c
    clr     DLARGEMSG
    movb    #$01, t3state
    rts

t3s8b:                                      ; Display on bottom line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s8bi

    ldx     #LARGEMSG
    ldaa    #$48                            ; Position after "TIME2 = "
    jsr     PUTCHAR_1ST
    bra     bh_t3s8b

t3s8bi:
    jsr     PUTCHAR

bh_t3s8b:
    tst     FIRSTCH
    beq     t3s8c
    clr     DLARGEMSG
    movb    #$01, t3state

t3s8c:
    rts

t3state9:                                   ; Display "ZERO MAGNITUDE ERROR"
message
    tst     F1_FLG
    bne     t3s9a
    tst     F2_FLG
    bne     t3s9b
```

```
    bra     t3s9c

t3s9a:                                          ; Display on top line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s9ai

    ldx     #ZEROMSG
    ldaa    #$08
    jsr     PUTCHAR_1ST
    bra     bh_t3s9a

t3s9ai:
    jsr     PUTCHAR

bh_t3s9a:
    tst     FIRSTCH
    beq     t3s9c
    clr     DZEROMSG
    movb    #$01, t3state
    rts

t3s9b:                                          ; Display on bottom line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s9bi

    ldx     #ZEROMSG
    ldaa    #$48
    jsr     PUTCHAR_1ST
    bra     bh_t3s9b

t3s9bi:
    jsr     PUTCHAR

bh_t3s9b:
    tst     FIRSTCH
    beq     t3s9c
    clr     DZEROMSG
    movb    #$01, t3state

t3s9c:
    rts

t3state10:                                      ; Display "NO DIGITS ENTERED" message
    tst     F1_FLG
    bne     t3s10a
    tst     F2_FLG
    bne     t3s10b
    bra     t3s10c
```

```
t3s10a:                                         ; Display on top line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s10ai

    ldx     #NOINPUT
    ldaa    #$08
    jsr     PUTCHAR_1ST
    bra     bh_t3s10a

t3s10ai:
    jsr     PUTCHAR

bh_t3s10a:
    tst     FIRSTCH
    beq     t3s10c
    clr     DNOINPUT
    movb    #$01, t3state
    rts

t3s10b:                                         ; Display on bottom line
    ldaa    FIRSTCH
    cmpa    #$01
    bne     t3s10bi

    ldx     #NOINPUT
    ldaa    #$48
    jsr     PUTCHAR_1ST
    bra     bh_t3s10b

t3s10bi:
    jsr     PUTCHAR

bh_t3s10b:
    tst     FIRSTCH
    beq     t3s10c
    clr     DNOINPUT
    movb    #$01, t3state

t3s10c:
    rts

;================================================================================
=====
;| TASK_4: Pattern_1
   |
;|   Controls the blinking pattern sequence for LED pair 1
   |
;|   Sequence: Green -> Off -> Red -> Off -> Both -> Off -> (repeat)
```

```
   |
;================================================================================
=====
TASK_4:
    ; State dispatcher
    ldaa    t4state
    lbeq    t4state0                        ; State 0: Initialization

    ; Check if timing is configured (TICKS_1 != 0)
    tstw    TICKS_1
    lbeq    t4state7                        ; TICKS_1 = 0, go to off state

    ; Route to pattern states
    ldaa    t4state
    deca
    lbeq    t4state1                        ; State 1: Green only
    deca
    lbeq    t4state2                        ; State 2: Both off
    deca
    lbeq    t4state3                        ; State 3: Red only
    deca
    lbeq    t4state4                        ; State 4: Both off
    deca
    lbeq    t4state5                        ; State 5: Both on
    deca
    lbeq    t4state6                        ; State 6: Both off
    rts                                     ; Undefined state

t4state0:                                   ; Initialize Port P for LED control
    bclr    PORTP, LED_MSK_1                ; Turn off both LEDs
    bset    DDRP, LED_MSK_1                 ; Configure pins as outputs
    movb    #$01, t4state                   ; Advance to first pattern state
    rts

t4state1:                                   ; Pattern: Green ON, Red OFF
    bset    PORTP, G_LED_1
    tst     DONE_1                          ; Check if timing period complete
    beq     exit_t4s1
    movb    #$02, t4state                   ; Advance to next pattern

exit_t4s1:
    rts

t4state2:                                   ; Pattern: Both OFF
    bclr    PORTP, G_LED_1
    tst     DONE_1
    beq     exit_t4s2
    movb    #$03, t4state

exit_t4s2:
```

```
    rts

t4state3:                                   ; Pattern: Green OFF, Red ON
    bset    PORTP, R_LED_1
    tst     DONE_1
    beq     exit_t4s3
    movb    #$04, t4state

exit_t4s3:
    rts

t4state4                                    ; Pattern: Both OFF
    bclr    PORTP, R_LED_1
    tst     DONE_1
    beq     exit_t4s4
    movb    #$05, t4state

exit_t4s4:
    rts

t4state5:                                   ; Pattern: Both ON
    bset    PORTP, LED_MSK_1
    tst     DONE_1
    beq     exit_t4s5
    movb    #$06, t4state

exit_t4s5:
    rts

t4state6:                                   ; Pattern: Both OFF (cycle complete)
    bclr    PORTP, LED_MSK_1
    tst     DONE_1
    beq     exit_t4s6
    movb    #$01, t4state                   ; Loop back to start

exit_t4s6:
    rts

t4state7:                                   ; No timing set - keep LEDs off
    bclr    PORTP, LED_MSK_1
    movb    #$06, t4state                   ; Return to state 6 when TICKS_1 is
set
    rts                                     ; Timing_1 will set DONE_1 = 1 when
ready

;=============================================================================
=====
;| TASK_5: Timing_1
   |
;|   Manages countdown timer for LED pair 1 pattern transitions
```

```
  |
;=================================================================================
=====
TASK_5:
    ; State dispatcher
    ldaa    t5state
    beq     t5state0                        ; State 0: Initialization

    ; Check if timing is configured
    tstw    TICKS_1
    beq     t5state2                        ; TICKS_1 = 0, idle state

    ldaa    t5state
    deca
    beq     t5state1                        ; State 1: Active countdown
    rts

t5state0:                                   ; Initialize timer
    clr     DONE_1
    movw    TICKS_1, COUNT_1                ; Load period value
    movb    #$01, t5state
    rts

t5state1:                                   ; Active countdown and
re-initialization
    tst     DONE_1                          ; Check if pattern consumed the done
flag
    beq     t5s1a                           ; Not consumed - continue counting
    movw    TICKS_1, COUNT_1                ; Pattern advanced - reload counter
    clr     DONE_1                          ; Clear done flag

t5s1a:
    decw    COUNT_1                         ; Decrement countdown
    bne     exit_t5s1                       ; Not zero - continue
    movb    #$01, DONE_1                    ; Zero - signal pattern to advance

exit_t5s1:
    rts

t5state2:                                   ; Idle state (no timing configured)
    movb    #$01, DONE_1                    ; Keep done flag set for
re-initialization
    rts

;=================================================================================
=====
;| TASK_6: Pattern_2
   |
;|   Controls the blinking pattern sequence for LED pair 2
   |
```

```
;|   Sequence: Green -> Off -> Red -> Off -> Both -> Off -> (repeat)
   |
;=============================================================================
=====
TASK_6:
    ; State dispatcher
    ldaa    t6state
    lbeq    t6state0

    ; Check if timing is configured
    tstw    TICKS_2
    lbeq    t6state7

    ; Route to pattern states
    ldaa    t6state
    deca
    lbeq    t6state1                        ; State 1: Green only
    deca
    lbeq    t6state2                        ; State 2: Both off
    deca
    lbeq    t6state3                        ; State 3: Red only
    deca
    lbeq    t6state4                        ; State 4: Both off
    deca
    lbeq    t6state5                        ; State 5: Both on
    deca
    lbeq    t6state6                        ; State 6: Both off
    rts

t6state0:                                   ; Initialize Port P for LED control
    bclr    PORTP, LED_MSK_2
    bset    DDRP, LED_MSK_2
    movb    #$01, t6state
    rts

t6state1:                                   ; Pattern: Green ON, Red OFF
    bset    PORTP, G_LED_2
    tst     DONE_2
    beq     exit_t6s1
    movb    #$02, t6state

exit_t6s1:
    rts

t6state2:                                   ; Pattern: Both OFF
    bclr    PORTP, G_LED_2
    tst     DONE_2
    beq     exit_t6s2
    movb    #$03, t6state
```

```
exit_t6s2:
    rts

t6state3:                                       ; Pattern: Green OFF, Red ON
    bset    PORTP, R_LED_2
    tst     DONE_2
    beq     exit_t6s3
    movb    #$04, t6state

exit_t6s3:
    rts

t6state4                                        ; Pattern: Both OFF
    bclr    PORTP, R_LED_2
    tst     DONE_2
    beq     exit_t6s4
    movb    #$05, t6state

exit_t6s4:
    rts

t6state5:                                       ; Pattern: Both ON
    bset    PORTP, LED_MSK_2
    tst     DONE_2
    beq     exit_t6s5
    movb    #$06, t6state

exit_t6s5:
    rts

t6state6:                                       ; Pattern: Both OFF (cycle complete)
    bclr    PORTP, LED_MSK_2
    tst     DONE_2
    beq     exit_t6s6
    movb    #$01, t6state                       ; Loop back to start

exit_t6s6:
    rts

t6state7:                                       ; No timing set - keep LEDs off
    bclr    PORTP, LED_MSK_2
    movb    #$06, t6state                       ; Return to state 6 when TICKS_2 is
set
    rts

;=============================================================================
=====
;| TASK_7: Timing_2
   |
;|   Manages countdown timer for LED pair 2 pattern transitions
```

```
  |
;=================================================================================
=====
TASK_7:
    ; State dispatcher
    ldaa    t7state
    beq     t7state0

    ; Check if timing is configured
    tstw    TICKS_2
    beq     t7state2

    deca
    beq     t7state1
    rts

t7state0:                                   ; Initialize timer
    clr     DONE_2
    movw    TICKS_2, COUNT_2
    movb    #$01, t7state
    rts

t7state1:                                   ; Active countdown and
re-initialization
    tst     DONE_2
    beq     t7s1a
    movw    TICKS_2, COUNT_2
    clr     DONE_2

t7s1a:
    decw    COUNT_2
    bne     exit_t7s1
    movb    #$01, DONE_2

exit_t7s1:
    rts

t7state2:                                   ; Idle state (no timing configured)
    movb    #$01, DONE_2
    rts

;=================================================================================
=====
;| TASK_8: Delay
   |
;|   Generates 1 millisecond system tick for timing tasks
   |
;=================================================================================
=====
TASK_8:
```

```
    ; State dispatcher
    ldaa    t8state
    beq     t8state0
    deca
    beq     t8state1
    rts

t8state0:                                   ; Initialization (none required)
    movb    #$01, t8state
    rts

t8state1:                                   ; Generate 1ms delay
    jsr     DELAY_1ms
    rts

;/-------------------------------------------------------------------------------
--\
;| Subroutines
 |
;\-------------------------------------------------------------------------------
--/

;-------------------------------------------------------------------------------
-
; PUTCHAR_1ST / PUTCHAR
;   Displays a null-terminated string on the LCD character-by-character
;
;   Entry point 1: PUTCHAR_1ST
;     Inputs:  X  = address of first character in string
;              A  = LCD cursor address
;     Clears FIRSTCH flag and sets cursor position
;
;   Entry point 2: PUTCHAR
;     Uses DPTR to continue displaying string from previous call
;     Automatically increments to next character
;
;   Exit: FIRSTCH = 1 when null terminator reached (string complete)
;-------------------------------------------------------------------------------
-
PUTCHAR_1ST:
    stx     DPTR                            ; Store string address
    jsr     SETADDR                         ; Set LCD cursor position
    clr     FIRSTCH                         ; Mark as not first character anymore

PUTCHAR:
    ldx     DPTR                            ; Load current character address
    ldab    0,X                             ; Read character
    beq     disp_done                       ; Null terminator - string complete
    incw    DPTR                            ; Advance to next character
    jsr     OUTCHAR                         ; Display current character
```

```
    rts

disp_done:
    movb    #$01, FIRSTCH                   ; Reset flag for next string
    rts

;---------------------------------------------------------------------------
-
; DELAY_1ms
;   Generates approximately 1 millisecond delay at 20 MHz bus clock
;   Uses busy-wait loop with 1412 iterations (0x0584)
;---------------------------------------------------------------------------
-
DELAY_1ms:
    ldy     #$0584                          ; Load loop counter

INNER:
    cpy     #0                              ; Check if counter expired
    beq     EXIT
    dey                                     ; Decrement counter
    bra     INNER

EXIT:
    rts

;---------------------------------------------------------------------------
-
; ASCII_2_BIN
;   Converts ASCII digit string in BUFFER to 16-bit binary value
;
;   Inputs:  BUFFER = array of ASCII digit characters ('0'-'9')
;            COUNT  = number of digits in BUFFER (1-5)
;
;   Outputs: X = binary value (0-65535)
;            A = error code:
;                0 = success
;                1 = value too large (overflow during multiplication)
;                2 = value is zero
;
;   Algorithm: Multiply-and-add (Horner's method)
;              result = 0
;              for each digit:
;                  result = result * 10 + digit
;
;   Stack usage: 3 bytes local storage
;                SP+0 = TEMP (digit counter)
;                SP+1:2 = RESULT (accumulated value)
;---------------------------------------------------------------------------
-
ASCII_2_BIN:
```

```
    ; Save registers
    pshb
    pshy

    ; Allocate local variables on stack
    des                                 ; SP+0 = TEMP
    des                                 ; SP+1 = RESULT (high byte)
    des                                 ; SP+2 = RESULT (low byte)

    ; Initialize local variables
    clr     0,SP                        ; TEMP = 0 (digit position)
    clrw    1,SP                        ; RESULT = 0

    ldx     #BUFFER                     ; Point to digit buffer

bcdloop:
    ; Load next digit
    ldaa    0,SP                        ; A = TEMP (current digit index)
    ldab    A,X                         ; B = BUFFER[TEMP] (ASCII digit)
    subb    #$30                        ; Convert ASCII to numeric (0-9)

    ; Add digit to result
    clra                                ; Clear high byte for 16-bit addition
    addd    1,SP                        ; D = D + RESULT
    bcs     overflow                    ; Carry indicates overflow
    std     1,SP                        ; Store updated result

    ; Check if all digits processed
    inc     0,SP                        ; TEMP++
    dec     COUNT                       ; Decrement remaining digit count
    beq     bcddone                     ; All digits processed

    ; Multiply result by 10 for next digit
    ldy     #10
    emul                                ; D = D * Y (with potential overflow
to Y)
    tsty                                ; Check if high word non-zero
    bne     overflow                    ; Overflow - value too large
    std     1,SP                        ; Store result
    bra     bcdloop                     ; Process next digit

bcddone:
    ; Check for zero value error
    cpd     #$0000
    beq     zero

    ; Success - transfer result to X
    tfr     D,X
    clra                                ; Error code = 0 (success)
    bra     exit
```

```
overflow:
    ldaa    #01                             ; Error code = 1 (too large)
    bra     exit

zero:
    ldaa    #02                             ; Error code = 2 (zero value)
    bra     exit

exit:
    ; Restore stack and registers
    ins
    ins
    ins
    puly
    pulb
    rts

;/---------------------------------------------------------------------------
--\
;| ASCII Messages and Constant Data
|
;\---------------------------------------------------------------------------
--/
TIME1:              DC.B    'TIME1 = ', $00
F1PRMPT:            DC.B    '      <F1> to update LED1 period', $00
TIME2:              DC.B    'TIME2 = ', $00
F2PRMPT:            DC.B    '      <F2> to update LED2 period', $00
BACKSPACE:          DC.B    $08, $20, $08         ; Backspace sequence: BS, space,
BS
ZEROMSG:            DC.B    'ZERO MAGNITUDE ERROR ', $00
LARGEMSG:           DC.B    'MAGNITUDE TOO LARGE ', $00
NOINPUT:            DC.B    'NO DIGITS ENTERED ', $00

;/---------------------------------------------------------------------------
--\
;| Interrupt Vectors
|
;\---------------------------------------------------------------------------
--/
                    ORG     $FFFE               ; Reset vector
                    DC.W    Entry

                    ORG     $FFCE               ; Port J Key Wakeup interrupt
vector
                    DC.W    ISR_KEYPAD
```